perp_sim: Environment (Phase 0)

1. What this is, in one page

perp_sim is a miniature crypto exchange that runs on a laptop, so we can run market-risk experiments that are impossible on a real exchange.

On a real exchange we cannot add 50 AI traders and watch what happens to the price. In perp_sim we can. We control who trades, what the outside price does and how much liquidity there is, and we can replay the exact same market with only one thing changed.

It copies the mechanics that make crypto "perpetual futures" markets dangerous:

- **Leverage**: betting with borrowed power. $10,000 at 10x controls a $100,000 position, so a 1% price move changes your money by 10%.
- **Margin**: the cash deposit that backs a bet, like a rental deposit.
- **Liquidation**: when a trader's losses eat most of the deposit, the exchange force-closes the position automatically.
- **Funding**: an hourly payment between buyers and sellers that keeps the exchange price close to the outside price.

A forced sale pushes the price down, and the lower price can force the next trader out. That chain reaction is a **cascade**, and it is the risk this project studies.

The package uses only Python's standard library, so nothing needs installing. The same seed always gives the same result.

What it provides for each phase

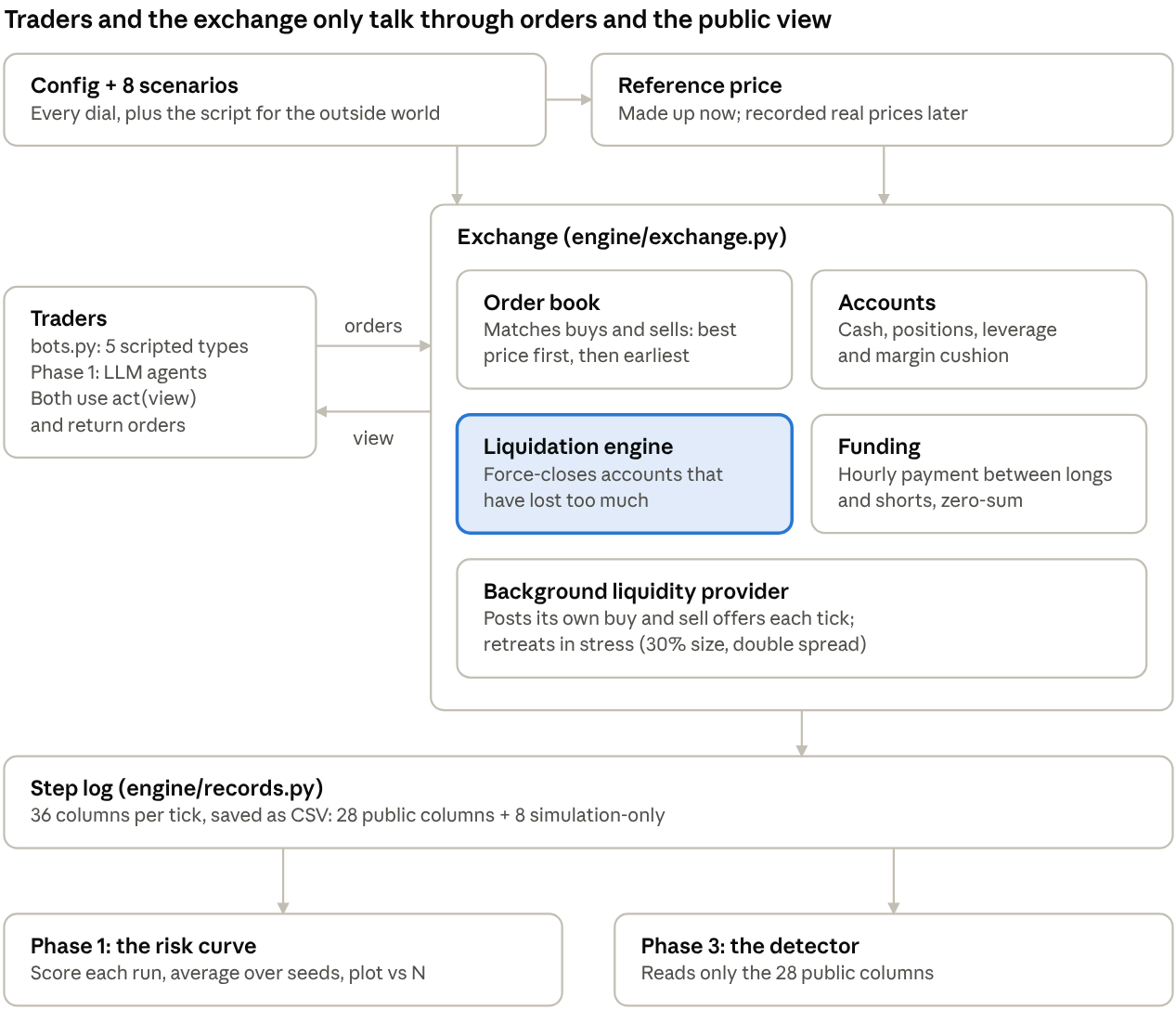
Phase	Question	What perp_sim provides
1. Risk discovery	How does risk change as the number of AI agents N grows?	The market the agents trade in, and a step-by-step log of every run
2. Intervention testing	Which rules (such as a lower leverage cap) reduce the risk?	Dials in one config file, and a switch that turns forced closing off for comparison
3. Detection	Can public signals warn us before a cascade?	Log columns split into public (what a real exchange shows) and simulation-only

What it is not yet

- It uses no real data. The outside price and the traders are generated.
- No number in it is calibrated. Every setting is a placeholder.
- It has no AI agents. Thirteen scripted bots stand in for them.
- Nothing it prints should be quoted as a finding about the real Bitcoin market.

2. The big picture: how the pieces connect

The exchange sits in the middle. Traders send it orders and get back a public view, a script sets the outside world, and everything it does is written to one log that Phases 1 and 3 both read.



architecture of perp_sim · 10 parts, who talks to whom

The runner (`run_demo.py` now, the Phase 1 runner later) drives the loop: each tick it asks every trader for orders, hands them to `Exchange.step()` and keeps the step log. The highlighted liquidation engine is the only part that can force a trade on a trader.

3. What happens in one simulated minute

The market runs in ticks. One call to `Exchange.step()` is one tick (one simulated minute by default), and it always does the same seven things in the same order.

1. **The outside price moves.** The reference price takes a small random step, sometimes a jump, and on one scheduled step a large shock.

2. **The background market refreshes.** Old orders expire. The exchange's own liquidity provider cancels its quotes and posts a fresh ladder of 5 buy and 5 sell offers around the current price. In stress scenarios, if the price moved more than 1% in the last 5 ticks, it retreats to 30% of its size and doubles its spread.
3. **Traders' orders arrive.** Each trader first looked at a `view` of the market (public information plus its own account) and returned a list of orders. The exchange shuffles them with a seeded random draw so nobody always goes first. Each order is shrunk if it would break the leverage cap, then matched in the order book.
4. **The liquidation engine runs.** Every account whose equity has fallen below 2.5% of its position value is force-closed, worst first. Each forced sale can push the price down and tip the next account over. The engine repeats this up to 25 rounds in one tick, and each account is attempted once per tick. Anything not filled is retried next tick.
5. **Funding is settled if an hour has passed.** Every tick the gap between exchange price and reference price is sampled. Each hour, the average gap sets a rate, and longs pay shorts (or the reverse). Every dollar one side pays, the other receives.
6. **Bad debt is recorded.** If a closed account owes more than its cash, nobody can collect, so the shortfall is logged as bad debt and the account is reset to zero.
7. **One row is written** to the step log: 36 columns covering prices, order-book state, trading, funding, liquidations and hidden risk measures.

One design choice is worth knowing. The outside price has its own random stream, separate from the one that shuffles orders. So for a given scenario and seed, the outside price path is identical whether 5 or 100 traders are present. That is what makes comparisons across N fair.

4. File by file

The package has three layers: the engine (the market rules), the bots (stand-in traders) and the demo (the script that runs them together). Each file below is explained in plain words, then in terms of what its code does.

```
perp_sim/
  README.md            how to run it
  run_demo.py          the demo script
  bots.py              five kinds of scripted trader
  engine/
    __init__.py        the list of things the engine exposes
    config.py          every dial and the Scenario definition
    orderbook.py       the order book and matching rule
    accounts.py        cash, position, leverage, margin
    index_price.py     the outside (reference) price
    scenarios.py       the 8 fixed scenarios
    records.py         the per-tick log columns
    exchange.py        the market loop that ties it all together
  tests/               41 automated checks
  output/              the CSV written by the demo
```

engine/config.py: the dials

This file holds every setting in one place, so changing the experiment never means hunting through code. `SimConfig` is a frozen dataclass: a settings object that cannot be changed by accident in the middle of a run. `Scenario` describes one fixed script for the outside world. **Every number here is a placeholder** until calibrated on real data.

Dial	Default	What it means
`n_steps`	120	How many ticks one run lasts
`step_minutes`	1.0	Real time one tick stands for. With `n_steps` it is how the demo scales up
`initial_price`	$100,000	Starting Bitcoin price (a round, roughly Bitcoin-sized number)
`tick_size`	$0.50	Smallest price move allowed
`lot_size`	0.001 BTC	Smallest order size allowed
`max_leverage`	20x	Per-account cap. Phase 2 tests lowering it
`maintenance_margin_rate`	2.5%	Equity must stay above this share of position value, or the account is liquidated
`initial_collateral`	$10,000	Starting cash per trader

Dial	Default	What it means
`taker_fee`, `maker_fee`	0.045%, 0.015%	Fees on traded value. Takers (who accept an offer) pay more than makers (who post one)
`funding_interval_minutes`	60	How often funding is paid
`funding_scale`	1/8	Share of the average price gap that becomes the hourly funding rate
`funding_interest_per_hour`	0.00125%	A small fixed baseline added to the funding rate
`funding_cap_per_hour`	4%	Funding can never exceed this
`bg_anchor_strength`	0.2	How hard the background market is pulled back toward the reference price each tick (0 = ignores it, 1 = snaps to it)
`bg_levels`, `bg_level_gap`	5, 0.3%	The background ladder has 5 price levels per side, 0.3% apart, so it reaches 1.5% from the price
`depth_band`	0.5%	"Depth" means resting size within 0.5% of the mid price
`near_liq_buffer`	5%	An account is "near liquidation" when its cushion is below 5%
`liquidations_enabled`	True	Turning it off gives the counterfactual "what if there were no forced closing"
`max_liquidation_rounds`	25	Safety limit on chain reactions inside one tick
`seed`	0	The number that fixes the randomness

Two computed values sit on `SimConfig`: `hours_per_step` (step minutes divided by 60) and `funding_interval_steps` (how many ticks make one funding interval). Funding timing, the scenario shock timing and the volatility per tick all follow from `n_steps` and `step_minutes`. That is why the demo can scale up by changing two numbers.

`Scenario` holds the outside world's script: the reference price's volatility per hour, how often it jumps and how big the jumps are, when the shock hits (as a share of the run) and how big it is, how much background liquidity there is, how tight its quotes are, and whether it retreats in stress.

engine/orderbook.py: who wants to buy or sell, and the matching rule

The order book is the live list of offers. This file stores them and decides which offers trade with which. The rule is the same as on a real exchange: the best price goes first, and at the same price the earlier order goes first. A trade always happens at the price of the order that was already waiting.

- **`Order`** holds one order: who sent it, buy or sell (`BUY = 1`, `SELL = -1`), the size, the limit price (or none for a market order), how much is still unfilled, and two flags. `tif` is "time in force": `GTC` may rest in the book, `IOC` (immediate-or-cancel) fills what it can now and the rest vanishes. `reduce_only` means the order may only shrink an existing position. An order can also carry an expiry step.
- **`Fill`** records one trade: the price, the size, and who was the taker (the order that arrived) and the maker (the order that was waiting).
- **`OrderBook`** keeps two dictionaries, bids and asks, each mapping a price to a queue of waiting orders (oldest first), plus sorted price lists so the best bid and best ask are found instantly.

The core is `submit(order)`:

1. Look at the best price on the opposite side. If a limit order's price does not reach it, stop.
2. If the waiting order belongs to the same account, cancel the waiting one instead of trading (self-trade protection).
3. Otherwise trade the smaller of the two sizes at the waiting order's price, and remove the waiting order if it is used up.
4. Repeat until the order is filled or the book runs out. A market order simply keeps going, which is how a large order "walks the book" to worse prices.
5. If a limit `GTC` order has a remainder, it rests in the book. An `IOC` remainder disappears.

Smaller helpers read or tidy the book: `round_price` (snap to the tick), `best_bid`, `best_ask`, `mid`, `cancel`, `cancel_account`, `expire`, `resting_qty`, `open_orders`, `top_size`, and `depth(band)`, which adds up the size resting within a band (for example 0.5%) of the mid price on each side. Ten tests cover: best price first, earlier first, partial

fills, IOC remainders vanishing, market orders walking the book, no trade without a price cross, self-trade prevention, expiry, cancelling an account, and depth counting only inside the band.

engine/accounts.py: one trader's money

An `Account` tracks a trader's cash and position, and does the margin arithmetic that decides when they get liquidated. Position is in BTC: positive is long (betting up), negative is short (betting down).

- **Fields:** `collateral` (cash, adjusted by realised profit, fees and funding), `position`, `entry_price` (the average price paid), `realized_pnl`, `fees_paid`, `funding_paid`, `n_liquidations`, `max_leverage`, and two flags (`liquidatable`, `background`).
- **`equity(mark)`** is cash plus unrealised profit at the current mark price.
- **`notional(mark)`** is the position's size in dollars.
- **`margin_buffer`** is how far equity sits above the maintenance requirement, as a share of position value. Negative means already liquidatable.
- **`is_breached`** is true when equity falls below 2.5% of position value. This is the liquidation trigger.
- **`liquidation_price`** is the price at which that would happen, holding everything else fixed.
- **`apply_fill`** updates the account for a trade. Adding to a position blends the entry price. Reducing realises profit or loss on the part closed. Flipping from long to short realises the old side and opens the new side at the fill price. Fees come out of cash.

The tests show why leverage matters: a 5x long is only liquidated after a price fall of about 18%, while a 19x long is liquidated after about 2.8%. Nine tests cover blending, realised profit, shorts, flips, fees, equity, the liquidation-price formula for longs and shorts, breach detection and that 5x versus 19x comparison.

engine/index_price.py: the outside price

This file produces the "reference price": what Bitcoin is worth outside our exchange. It does two jobs. Funding is measured against it, and the background liquidity provider keeps its quotes loosely tied to it.

It is made up on purpose, because Phase 0 is a controlled laboratory. Each tick, `IndexPriceProcess.next(step)` computes one log-return (a percentage-style change) from three parts:

1. A random wiggle sized by the scenario's hourly volatility, scaled to the tick length, with a small correction so the price has no built-in drift.
2. Occasionally a random jump. The chance per tick is the jump rate per hour times the tick length in hours, and the size is drawn from the scenario's jump mean and spread.
3. On exactly one scheduled step, the shock: a one-off move of the scenario's shock size (for example -6%).

It uses its own random stream, so the price path never depends on who is trading. It is also swappable: anything with a `next(step)` method that returns the next price will plug in, such as a replay of recorded real prices.

engine/scenarios.py: the 8 fixed scenarios

The project plan calls for 8 fixed scenarios shared by every condition. This file builds them as every combination of three yes/no factors (2 x 2 x 2 = 8). The plan does not say which factors, so these are a placeholder design that needs team confirmation.

Factor	Option A	Option B
Market state	**normal**: volatility 0.5% per hour, 0.2 jumps per hour, shock -0.8%, quotes 1 bp from the price, makers never retreat	**stress**: volatility 1.2% per hour, 1.0 jumps per hour (average -0.1%), shock -6%, quotes 2 bp from the price, makers retreat when the price moves fast
Background depth	**thick**: 12 BTC per side	**thin**: 5 BTC per side
Shock timing	**early**: 35% of the way through the run	**late**: 65% of the way through the run

Scenarios are named `regime-depth-timing`, such as `stress-thin-early`, and stored in the `SCENARIOS` dictionary. `Scenario.shock_step(n_steps)` turns "35% of the run" into a step number, so the shock lands in the right place whatever the run length. Edit `build_scenarios()` to change the design; nothing else depends on it.

engine/records.py: what is written down every tick

`StepRecord` is one row of the log, with 36 columns split into two groups on purpose. The 28 `PUBLIC_FIELDS` are things a real exchange shows everyone. The 8 `SIM_ONLY_FIELDS` are things only the simulation can know, such as who is close to liquidation. The Phase 3

detector may only read the public group, so its result could carry over to a real exchange. `write_csv` saves a list of records as a spreadsheet-style file.

Group	Columns	What they tell us
Time	`step`, `minute`	The tick number and simulated minutes elapsed
Prices	`index_price`, `last_price`, `mark_price`, `mark_basis`, `price_return`, `index_return`	The outside price, the latest trade, the margin-judging price, how far apart the exchange and outside prices are, and the change since last tick
Order book	`best_bid`, `best_ask`, `spread`, `spread_rel`, `depth_bid`, `depth_ask`, `top_bid_size`, `top_ask_size`	The cost of trading now and the cushion available to absorb forced sales
Trading	`volume`, `n_trades`, `signed_volume`, `order_flow_imbalance`, `open_interest`	How much traded, whether buyers or sellers drove it (-1 to +1), and the total size of open positions
Funding	`funding_rate_est`, `last_funding_rate`, `funding_settled`	The hourly crowding fee, now and at the last payment
Liquidations	`liq_count`, `liq_notional`, `liq_qty_long`, `liq_qty_short`	Forced closes this tick: how many, how large, forced selling (longs) versus forced buying (shorts)
Simulation-only: agents	`n_agent_orders`, `n_agent_buys`, `n_agent_sells`, `n_rejected_orders`	How many orders were sent, how many were buys or sells (herding), and how many the exchange refused

Group	Columns	What they tell us
Simulation-only: hidden risk	`oi_near_liq_share`, `min_margin_buffer`, `bad_debt_total`, `bg_liquidity_factor`	How much forced selling is "loaded", the most fragile position's cushion, unrecoverable losses, and how much of the background market has pulled back

engine/exchange.py: the market itself

This is the heart of the package. `Exchange` owns the order book, all accounts, the reference price and the logs, and runs the seven-step tick described in section 3. Traders never touch its internals. They only receive a `MarketView` and send back `OrderRequest` or `CancelAll` objects, which is exactly the contract future AI agents will use.

What goes in and out (small data classes at the top of the file)

- **`OrderRequest`**: what a trader sends. Account, side, size in BTC, a limit price (none means market order), `tif` (`GTC` or `IOC`), `reduce_only`, and an optional `ttl_steps` after which a resting order is removed.
- **`CancelAll`**: cancel every resting order of one account.
- **`MarketView`**: what a trader may see. The step and minute, reference, last and mark prices, best bid, best ask and mid, bid and ask depth, estimated and last funding rate, ticks until the next funding payment, open interest, liquidations in the previous tick (count and dollar value), the last 61 prices, and its own account.
- **`OwnView`**: the trader's own account: cash, equity, position, entry price, leverage, margin cushion, liquidation price and number of open orders.
- **`LiquidationEvent`**: one forced close, recorded for analysis: step, account, which side was closed, size requested and filled, average price, dollar value and the mark price before.

Setup. `Exchange.__init__` creates two separate random streams (one for the reference price, one for the order of arrival), an empty order book, the price history, the logs (`records`, `order_log`, `liquidation_log`), and one special account called `__background__`. That account is the exchange's own liquidity provider. It has effectively unlimited cash and can never be liquidated. `add_account` registers each trader with the default $10,000 and the leverage cap.

Prices and funding helpers

- **`mark_price()`** is the median of three numbers: the reference price, the last trade and the book's mid price. A median means one odd number cannot move it, but a falling book does drag it down, which is how forced selling can feed on itself.
- **`_funding_rate_now()`** is the average price gap since the last payment times 1/8, plus the small baseline, limited to plus or minus 4% per hour.
- **`view(account_id)`** assembles the `MarketView` from public information plus that one account.

Handling an order (**`_handle_request`**)

1. Round the size down to the lot size and the price to the tick.
2. A reduce-only order is refused if the trader has no position or is on the same side, and is capped at the size of the position.
3. Any other order is passed through **`_clip_for_leverage`**, which shrinks it so that even if everything the trader has resting were filled, the account stays under the leverage cap. Orders that shrink to nothing are counted as rejected.
4. The order is logged and sent to the book. **`_apply_fills`** then charges the taker and maker fees, updates both accounts and sets the last price and the tick's volume counters.

The background liquidity provider (**`_requote_background`**)

Every tick it cancels its old quotes and posts a new ladder. The centre is `last price + anchor strength x (reference price - last price)`, so it sits between where the exchange last traded and the outside price. Its half-spread comes from the scenario. Each level is the scenario's depth divided by the number of levels, and levels are `bg_level_gap` apart. In stress scenarios, if the price moved more than 1% in the last 5 ticks, it keeps only 30% of its size and doubles its spread. The 30% is logged as `bg_liquidity_factor`. Its size is the same whatever the number of AI agents N, so depth is a fixed backdrop rather than something that scales with the crowd.

The liquidation engine (**`_run_liquidations`**, **`_liquidate`**)

It finds every liquidatable account whose equity is below 2.5% of position value at the current mark price, and works through them worst first. For each one it cancels the account's resting orders, then sends a reduce-only market order of the whole position (a sell for a long, a buy for a short) that fills as much as the book allows. The result is logged as a `LiquidationEvent`. After each forced sale the mark price is recomputed, since the sale may have pushed the next account over the line. The loop runs up to 25 rounds, and

each account is tried once per tick. If the book is too thin to take the whole position, the unfilled part stays open and is retried next tick. `liquidations_enabled=False` skips this whole step, which gives the "no forced closing" counterfactual.

Funding and bad debt

- **`_sample_premium`** records the gap between the book's mid price and the reference price each tick.
- **`_settle_funding`**, once per funding interval, charges each account `position x mark price x rate x hours`. Longs pay when the rate is positive, shorts receive, and the totals cancel exactly.
- **`_sweep_bad_debt`** records any negative balance left after an account is flat as bad debt and resets it to zero. Real exchanges cover this with an insurance fund or auto-deleveraging, which is not modelled yet and is a possible Phase 2 intervention.

Logging (`_make_record`) builds the 36-column row. For the hidden risk columns it totals position value across all non-background accounts, finds how much of it belongs to accounts with a cushion under 5%, and records the smallest cushion.

Twenty-two tests in `test_exchange.py` cover this file, listed in section 5.

engine/ `__init__.py` : the front door

This short file re-exports the engine's public names so other code can write `from engine import Exchange, SimConfig, SCENARIOS, OrderRequest` instead of knowing which file each lives in. It contains no logic; `__all__` lists everything that is meant to be used from outside.

bots.py: five kinds of scripted trader

The bots exist only to test the exchange. They are simple rule-followers with no AI and no cost, and they follow the same contract the Phase 1 AI agents will: `bot.act(view)` receives a `MarketView` and returns a list of orders.

The base class `Bot` gives each bot its own seeded random generator (so runs are repeatable) and an `act_prob`, the chance it does anything on a given tick. Its `act` method rolls that chance and then calls `decide`, which each type overrides. Two helpers are shared:

- **`_go_to(view, target)`** trades toward a target position using an aggressive limit order that may cross the spread but never pays more than 0.3% beyond the best quote. It ignores adjustments under 0.01 BTC.
- **`_size_for(view, leverage)`** converts a leverage figure into BTC: `leverage x equity / mark price`.

Bot	What it does	Why it is in the test
`CrowdBot`	Builds a leveraged long over 20 ticks, then holds it. Acts on 70% of ticks.	Deliberately artificial. Several together create the crowded leveraged longs a cascade needs. In Phase 1 the question is whether AI agents do this on their own.
`TrendBot`	Looks at the price change over the last 10 ticks. If it moved at least 0.15%, goes long (rose) or short (fell) at 4x. Acts on 50% of ticks.	Momentum traders amplify moves.
`ContrarianBot`	Bets against a move of at least 0.3% over 10 ticks, at 3x.	Supplies the opposite view, which can absorb selling.
`ValueBot`	Compares the exchange price with the reference price. Buys when the exchange is more than 0.15% below it, sells when above, and goes flat when within 0.05%.	Pulls the exchange back toward the outside price.
`MakerBot`	Each tick cancels its quotes and posts one buy and one sell 0.03% either side of the mid price (0.2 BTC each). Stops quoting a side once its inventory passes 1 BTC.	Adds some resting liquidity from a participant rather than the background provider.

run_demo.py: the demo script

This script wires the engine and the bots together, runs the experiments and prints the results as text. It is also the template for the Phase 1 runner, because its main loop is the loop agents will sit inside. Walking through it top to bottom:

- **`CROWD_LEVERAGE = [6, 10, 14, 18]`**: the crowd bots use a spread of leverage, as real traders do.

- **`build_population(seed, crowd_copies=1)`** creates the traders: 4 crowd bots per copy, 4 trend bots (leverage 4, 4, 6, 6), 2 contrarian, 2 value and 1 maker. With one copy that is 13 bots. `crowd_copies=3` triples the crowd.
- **`run(scenario_name, seed, n_steps, step_minutes, liquidations, crowd_copies, **config_overrides)`** is the key function. It builds a `SimConfig` and an `Exchange`, creates the bots, registers an account for each, and then loops `n_steps` times: every bot calls `act(exchange.view(its_account))`, all returned orders are gathered into one list, and `exchange.step(actions)` advances the market. It returns the finished exchange. Extra keyword arguments override any dial, which is how the experiments change `bg_anchor_strength`.
- **`max_drawdown(prices)`** finds the worst fall from a previous peak.
- **`summarize(exchange)`** turns one finished run into a handful of risk numbers: max drawdown, how far the reference price ended from its start, number of liquidation orders, accounts liquidated, steps with liquidations, the largest single-tick forced sale, the step of the first liquidation, **`dip_below_ref`** (the deepest the exchange price fell below the reference price), bad debt, the highest near-liquidation share, the largest funding rate and the thinnest bid depth.
- **`pct()`** is a small formatting helper.
- **`main()`** runs four things in order:
  1. A detailed run of `stress-thin-early` (seed 1) with a text table of the ticks around the shock, saved to `output/demo_stress-thin-early_seed1.csv`.
  2. **The counterfactual test**: the same run with forced closing switched off, and the deepest dip below the reference price compared. This isolates the damage caused by forced selling.
  3. A table of all 8 scenarios, each averaged over 8 seeds, with liquidations on and off.
  4. A dial experiment: pull-to-reference strength (0.5, 0.2, 0.1) against crowd size (x1, x3), to show what decides whether a cascade forms.

We use `dip_below_ref` rather than "how far the price fell after the first liquidation" because the second measure mostly reflected the price converging toward the reference, and was misleading. The counterfactual comparison is the honest one.

Other files

- **`README.md`**: how to run the demo and the tests, a one-line description of each file, and the placeholder warnings.

- **`output/demo_stress-thin-early_seed1.csv`**: the 120-row step log from the detailed run, with all 36 columns.

5. How we know it works

41 automated tests pass. They check that the exchange follows its own rules, not that it matches the real world. Run them with `python -m unittest discover -s tests -t .` (about 0.05 seconds).

Test file	Tests	What it checks
`test_orderbook.py`	10	Best price first; earlier order first at the same price; partial fills rest the remainder; IOC remainders vanish; market orders walk the book; no trade when prices do not cross; self-trade prevention; expiry; cancelling an account; depth counts only inside the band
`test_accounts.py`	9	Adding blends the entry price; reducing realises profit; shorts profit when the price falls; flipping side; fees come out of cash; equity follows price; the liquidation price is where equity meets the requirement; breach detection; 5x needs a far bigger move than 19x
`test_exchange.py`: leverage	5	Orders are clipped to the cap; repeated orders cannot exceed it; reduce-only cannot open or grow a position or flip it; trades are zero-sum across accounts
`test_exchange.py`: funding	7	Longs pay when the exchange is above the reference price and shorts pay when below; funding is zero-sum; the payment matches the formula; the cap works; it settles once an hour at 1-minute steps and follows the step length at 5-minute steps
`test_exchange.py`: liquidation	5	A 15x long is force-closed after a shock and a 3x long survives; liquidations appear in the public log; an unfilled forced sale is retried; bad debt is recorded

Test file	Tests	What it checks
`test_exchange.py`: repeatability	3	Same seed gives identical logs; a different seed gives a different path; the reference price path is identical whatever the number of traders
`test_exchange.py`: scenarios	2	There are 8 distinct scenarios, 4 normal; shock timing scales with run length

Two more checks were run outside the test suite. The demo shows no liquidations in the four normal scenarios and about three accounts liquidated in the four stress scenarios, as expected. And a scaling run with 100 traders took about 0.2 seconds for 100 one-minute ticks and about 0.7 seconds for 300 five-minute ticks, with funding landing every 12 ticks at the longer step.

What the tests do not prove: that any number is realistic. They show the machinery is correct, not that its settings match a real exchange.

6. What the demo found, and what it does not prove

The demo shows that a liquidation cascade can happen in the engine, but only when two conditions line up: a large crowd of leveraged longs, and a market that is not quickly pulled back to the outside price. It does not show that AI agents cause crashes.

All figures below come from placeholder settings and scripted bots, averaged over 8 seeds unless stated.

- **Normal scenarios:** nobody is liquidated in any run. The exchange price dips about 0.24% to 0.29% below the reference price from ordinary noise.
- **Stress scenarios (a 6% shock):** about 3 accounts are liquidated in every run. With the default crowd of 4 leveraged long bots, forced selling adds little in the thick-liquidity scenarios. In the thin ones the dip below the reference price is -0.74% with liquidations versus -0.59% without (early shock), and -0.92% versus -0.56% (late shock).
- **One detailed run** (`stress-thin-early`, seed 1): the shock hits at tick 42 and the reference price drops from about $100,664 to $94,531. The exchange price lags down over the next four ticks ($99,301, $98,387, $97,648, $95,792) as bid depth falls from 2.2 BTC to 0. Liquidations hit on ticks 45 and 46, and in total 5 forced orders close 3 accounts. In that single run the deepest dip below the reference price was -0.56% with or without liquidations, so there was no amplification.

What decides whether forced selling feeds on itself. The last experiment varies two dials in the `stress-thin-early` scenario. The numbers are the deepest dip below the reference price, with forced closing on versus off:

Pull back to reference price	Crowd size	Accounts liquidated	Dip, liquidations on	Dip, liquidations off
0.5 (strong)	x1 (4 crowd bots)	3.1	-1.03%	-0.62%
0.5 (strong)	x3 (12 crowd bots)	9.5	-2.28%	-0.49%
0.2 (default)	x1	3.0	-0.74%	-0.59%
0.2 (default)	x3	9.9	-4.83%	-0.81%
0.1 (weak)	x1	2.8	-0.91%	-0.69%
0.1 (weak)	x3	10.0	-6.61%	-0.64%

Reading it: with a small crowd, forced selling barely matters. With three times the crowd and a weak pull back to the reference price, liquidations turn a dip of about 0.6% into 6.6%. Early in the build we found that with a background book reaching only 0.1% from the price, no cascade could form however large the forced sales were. We therefore widened the book's reach to 1.5% and weakened the pull. These two dials, the reach of the background book and its pull to the reference price, are the first things to calibrate on real data.

What this does not prove

- The crowd bots are artificial. They were built to create crowded leveraged longs so the mechanism could be tested. Whether real AI agents do this on their own is exactly the Phase 1 question.
- Every number is a placeholder, and 8 seeds is a small sample.
- The background liquidity is the same size at every N. That is a design choice, not a fact about markets.
- With a 5x leverage cap and 2.5% maintenance margin, a trader is only liquidated after a fall of about 18%, so cascades may be rare at the plan's default cap.

7. First look at real data (10 Oct 2025 crash hour)

Real data from the first hour of the 10 October 2025 crash shows our placeholders are off by large factors in four places: volatility in stress, the shape of the shock, spreads, and above all order-book size. The data is one hour of BTCUSDT trades (466,889 trades, 25,450 BTC) and order-book updates (7.9 million), 20:00 to 21:00 UTC, taken from the HFT data platform. Bitcoin fell from $116,605 to $114,187 in that hour, with a low of $112,800. According to CoinGecko's write-up, the violent phase began about 20:50 and ran to about 21:30, so this hour holds only the first 10 minutes of it.

	Our placeholder	Real, calm part (20:00 to 20:49)	Real, crash part (20:50 to 21:00)
Hourly price volatility	0.5% normal, 1.2% stress	0.44%	3.1%
Shape of the drop	One instant -6%	Gentle drift (worst 10 minutes -0.29%)	A slide of -2.3% over 10 minutes, with a dip to -3.45%
Gap between best buy and best sell	1 to 2 bp each side	0.009 bp (one tick, $0.10)	Typically 0.1 bp; 95th percentile 3 bp; worst 19 bp
Buy offers within 0.5% of price	5 BTC (thin) or 12 BTC (thick)	About 1,080 BTC	About 1,060 BTC on average; lowest 187 BTC
Sell offers within 0.5% of price	Same	About 1,170 BTC	About 770 BTC
Buy offers within 0.1% of price	Not modelled	About 225 BTC	About 155 BTC; lowest 5 BTC
Trading volume per minute	Not modelled	About 100 BTC	About 2,000 BTC (20x); peak near 5,000 BTC

What it means for the simulator

1. **Scale.** The real book holds about 1,100 BTC (roughly $125M) per side near the price, while our bots' forced sales are about 1.5 BTC. On a real-sized book our agents would never move the price. What matters is the size of the agents relative to the book, which is a team decision (section 9).

2. **Shock shape.** Real selling arrived minute after minute with heavy sell imbalance, not as one instant jump. The shock should be a fast multi-minute slide, or a replay of real prices.
3. **Spreads and liquidity.** Spreads are almost always one tick but blow out in the crash, and liquidity near the price drains. Both are candidate Phase 3 signals.

Caveats on the data

- The order-book file contains only updates, no starting snapshot. The book takes about one minute to fill, so the first minute is ignored.
- The recording lost 18 messages, all after 20:50, which left stale prices in the rebuilt book. I removed them using the trade prices, so the crash-minute depth figures are approximate and probably understate how thin the book got.
- 1,146 trades flagged `INSURANCE_FUND` appear only from 20:55, with price and quantity blank. They look like liquidation activity but carry no size, so they cannot calibrate forced-selling volume. The platform's liquidations dataset, if it has one, is needed.
- There is no funding-rate or mark-price data yet, and no calm comparison hour.

Nothing from this data has been put into the simulator yet.

8. How this plugs into Phase 1, 2 and 3

Phase 1 swaps the scripted bots for LLM agents and adds a runner around the exchange. The exchange does not change; the contract between traders and market stays exactly as it is.

Phase 1: risk discovery by number of agents

The contract is `agent.act(view)` returns a list of orders. A bot does it with a rule. An LLM agent will do it like this:

1. Turn the `MarketView` into a short text prompt: prices, spread, depth, funding, open interest, recent liquidations, the last 61 prices and the agent's own position, cash and leverage. It contains only public information plus the agent's own account.
2. Call the model and ask for a decision in a fixed format.
3. Parse the reply into an `OrderRequest` or `CancelAll`. An unreadable reply becomes "do nothing".

The **Phase 1 runner**, which is the next thing to build, wraps the loop that `run_demo.py` already has:

1. For each scenario (8), each agent count N and each seed, create an `Exchange`. The reference price path is identical across N for a given scenario and seed.
2. Each tick, take every agent's view before anyone acts, so decisions are simultaneous. Call all N agents at the same time, collect the orders and call `exchange.step(actions)`.
3. Save the step log (`records`), the order log, the liquidation log and each agent's stated reasoning.
4. Reduce each run to risk scores. `summarize()` in `run_demo.py` is the starting point: deepest dip below the reference price, total forced selling, thinnest depth, whether a cascade occurred, whether bad debt appeared.
5. Average the scores across seeds and scenarios for each N. **That average plotted against N is the risk curve**, and its bend is the finding.

Things that change because of this:

- **Cost scales with the number of calls**: N agents x ticks per run x scenarios x seeds. An earlier estimate was about 364 input and 70 output tokens per call. Tick length and run length are therefore cost dials, and the token estimator must be updated for the new call structure.
- **N is just the number of accounts.** Nothing in the engine limits it; 100 scripted agents run in a fraction of a second, so the engine is never the bottleneck. The model calls are.
- **Agent mix is a team decision.** Do all agents share one prompt, or do they get different personas and risk appetites? Identical agents may herd more, which is itself a finding.

Phase 2: intervention testing

Most interventions are already dials in `config.py`: the leverage cap (`max_leverage`), the maintenance margin (`maintenance_margin_rate`), the funding settings, and the on/off switch for forced closing. Rerun the same scenarios and seeds with one dial changed and compare the curves. `n_rejected_orders` shows how often a cap actually blocked an agent. Auto-deleveraging, an insurance fund, circuit breakers and position limits are not built yet.

Phase 3: detection and prediction

The detector may read only the 28 public columns, such as spread, depth, order-flow imbalance, open interest, funding and liquidation counts. The simulation-only columns, such as `oi_near_liq_share` and `bad_debt_total`, supply the labels ("danger is coming") but are never inputs. This keeps a trained detector transferable to a real exchange.

What must happen before Phase 1 results mean anything

- Calibrate the settings on real data (volatility, spreads, depth, how liquidity retreats).
- Decide how large the agents are relative to the book (section 9).
- Make the shock a realistic multi-minute slide, or replay real prices as the reference price.
- Add the LLM agent wrapper, the runner, the risk scoring and the aggregation, and update the cost estimator.

9. Open decisions for the team

Seven decisions need the team before Phase 1 results can be trusted. The first one blocks the most work.

#	Decision	Options	Recommendation
1	How big are the agents relative to the real order book?	A: a mini-market, with book depth scaled so that at the largest N the agents are 5 to 10% of it. B: the full real size (about 1,100 BTC per side), where $10,000 agents would be invisible	A. It keeps the real shapes (thinning, wider spreads) and is the only setup where agent crowding can plausibly cause a cascade
2	Which 8 scenarios?	The current placeholder is market state x depth x shock timing. Alternatives could vary volatility, liquidity and shock size separately	Confirm with the plan and the supervisor; nothing else depends on this choice
3	Should background liquidity grow with N?	Now it is the same at every N, so depth is a fixed backdrop	Keep it fixed so that changes across N come from the agents
4	What leverage cap should the baseline use?	The plan's default is 5x, but at 5x a trader survives a fall of about 18%, so cascades may be very rare	Test the baseline at several caps, so Phase 2 has a range to work with

#	Decision	Options	Recommendation
5	Which risk scores form the curve?	Proposed: deepest dip below the reference price; total forced selling; thinnest depth; whether a cascade happened; whether bad debt appeared	Use the dip and the cascade flag as the headline pair
6	Synthetic or real reference price?	A made-up path with a scheduled shock, or a replay of a recorded real path such as the 10 Oct 2025 crash	Replay real prices for the stress scenarios; keep synthetic for normal
7	One agent prompt, or personas with different risk appetites?	Identical agents may herd more	Start with one prompt, then add personas as a Phase 1 comparison

Also outstanding

- **More real data**: the 21:00 hour of trades and order book, a calm hour, and liquidations and funding data if the platform has them.
- **LLM hosting account** (such as Groq or DeepInfra) is not set up yet.
- **Citations in the plan** were flagged earlier as unverified.

10. Glossary

Plain-words definitions of every term used above.

Term	Meaning
BTC	Bitcoin, the coin being traded. In the simulation it costs about $100,000
Perpetual future	A bet on the price of BTC going up or down, without owning any, with no expiry date
Long / short	Betting the price will rise / fall

Term	Meaning
Leverage	Betting with borrowed power: 10x means a $10,000 deposit controls a $100,000 position
Margin (collateral)	The cash deposit that backs a bet
Equity	Cash plus unrealised profit or loss
Maintenance margin	The minimum equity, as a share of position value (2.5% here), below which the exchange force-closes you
Liquidation	The exchange's automatic forced closing of a position that has lost too much
Cascade	Forced sales push the price down, which forces more sales, in a chain reaction
Bad debt	A loss bigger than the trader's deposit that nobody can collect
Reference (index) price	The "true" outside Bitcoin price the exchange is pulled toward
Last price	The price of the most recent trade on our exchange
Mark price	The price used to judge margin: the median of reference price, last price and book mid
Funding	An hourly payment between longs and shorts that keeps the exchange price near the reference price
Order book	The live list of waiting buy offers (bids) and sell offers (asks)
Spread	The gap between the best buy offer and the best sell offer; the cost of trading now
Depth	The total size waiting near the price; the cushion that absorbs big orders
Tick (step)	One advance of the simulation clock; one minute by default
Limit order / market order	An order with a price limit / an order that takes whatever is available

Term	Meaning
IOC / GTC	Immediate-or-cancel (fill now or vanish) / good-till-cancelled (may rest in the book)
Reduce-only	An order that may only shrink an existing position
Maker / taker	The trader whose offer was waiting / the trader who accepted it
Open interest	The total size of all open long positions
Order-flow imbalance	Buyer-driven minus seller-driven volume, divided by total volume (-1 to +1)
Seed	The number that fixes the randomness; the same seed reproduces the same run
Scenario	One of 8 fixed scripts for the outside world
Counterfactual	The same run with one thing switched off (here, forced closing) to measure its effect
bp (basis point)	One hundredth of one percent: 100 bp = 1%
Calibration	Setting the simulator's numbers from real market data
Risk curve	Average risk score plotted against the number of agents N